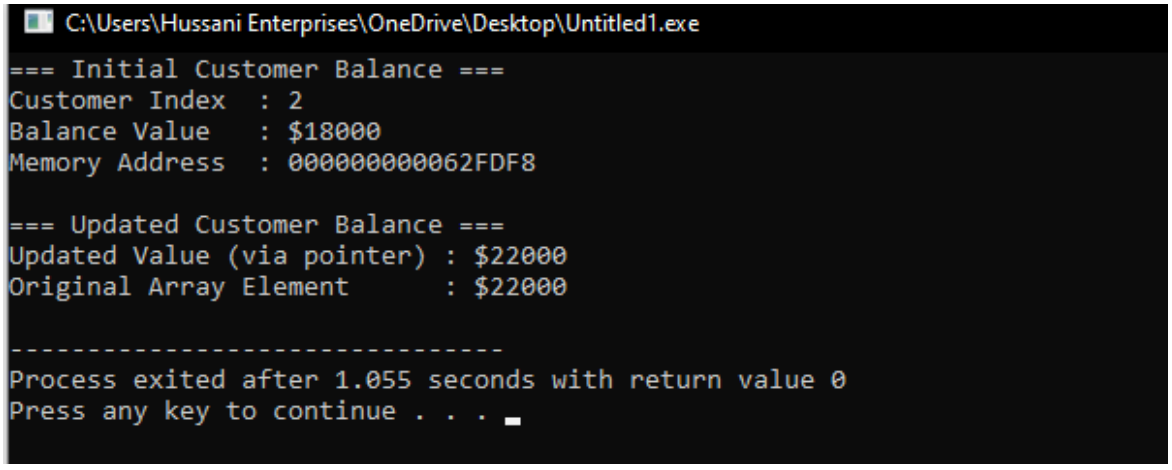
Pf in lab task 26k-3147

Q1:

```
#include <stdio.h>

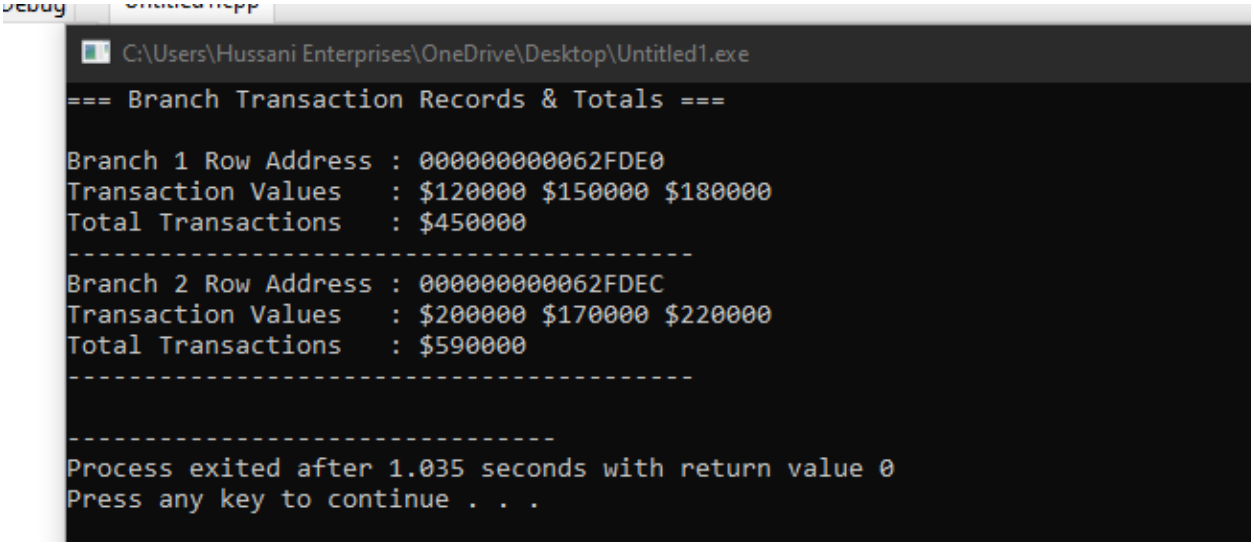
int main() {
   int balance[5] = {15000, 25000, 18000, 32000, 12000};
   int customerIndex = 2;
   int *ptr = &balance[customerIndex];
   printf("=== Initial Customer Balance ===\n");
   printf("Customer Index  : %d\n", customerIndex);
   printf("Balance Value   : $%d\n", *ptr);
   printf("Memory Address  : %p\n\n", (void *)ptr);
   int newAmount = 22000;
   *ptr = newAmount;
   printf("=== Updated Customer Balance ===\n");
   printf("Updated Value (via pointer) : $%d\n", *ptr);
   printf("Original Array Element      : $%d\n", balance[customerIndex]);
   return 0;
}
```



Q2:

```
#include <stdio.h>

int main() {
    int transactions[2][3] = {
        {120000, 150000, 180000},
        {200000, 170000, 220000}
    };
    int (*ptr)[3] = transactions;
    printf("=== Branch Transaction Records & Totals ===\n\n");
    for (int i = 0; i < 2; i++) {
        int totalTransactions = 0;
        printf("Branch %d Row Address : %p\n", i + 1, (void *)(ptr + i));
        printf("Transaction Values   : ");
        for (int j = 0; j < 3; j++) {
            int currentVal = *(*(ptr + i) + j);
            printf("$%d ", currentVal);
            totalTransactions += currentVal;
        }
        printf("\nTotal Transactions   : $%d\n", totalTransactions);
        printf("----------------------------------------\n");
    }
    return 0;
}
```



Q3:

```
#include <stdio.h>


int main() {
   int branch1[] = {15000, 23000, 32000};
   int branch2[] = {12000, 21000, 27000, 35000};

   int size1 = sizeof(branch1) / sizeof(branch1[0]);
   int size2 = sizeof(branch2) / sizeof(branch2[0]);

   int *branchPtrs[2] = {branch1, branch2};

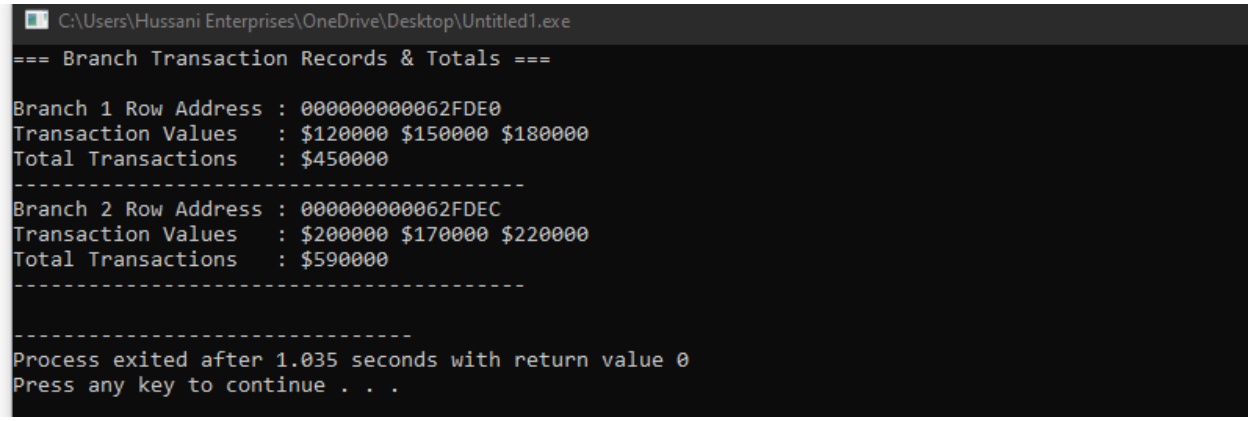
   printf("=== Branch 1 Accounts ===\n");
   int total1 = 0;
   for (int i = 0; i < size1; i++) {
      int *currentPtr = branchPtrs[0] + i;
```

```
        printf("Customer %d -> Balance: $%d | Address: %p\n", i + 1, *currentPtr, (void
*)currentPtr);
        total1 += *currentPtr;
    }
    printf("Total Balance (Branch 1) : $%d\n\n", total1);

    printf("=== Branch 2 Accounts ===\n");
    int total2 = 0;
    for (int i = 0; i < size2; i++) {
        int *currentPtr = branchPtrs[1] + i;

        printf("Customer %d -> Balance: $%d | Address: %p\n", i + 1, *currentPtr, (void
*)currentPtr);
        total2 += *currentPtr;
    }
    printf("Total Balance (Branch 2) : $%d\n", total2);

    return 0;
}
```



Q4:

```
#include <stdio.h>

int calculateTotal(int *arr, int size) {
   int total = 0;
   for (int i = 0; i < size; i++) {

      total += *(arr + i);
   }
   return total;
}

int main() {

   int branch1[] = {15000, 23000, 32000};
   int branch2[] = {12000, 21000, 27000, 35000};

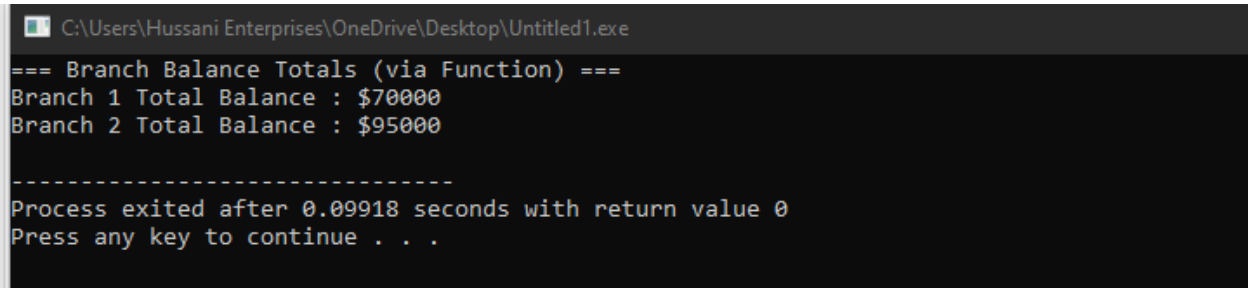
   int size1 = sizeof(branch1) / sizeof(branch1[0]);
   int size2 = sizeof(branch2) / sizeof(branch2[0]);

   int totalBranch1 = calculateTotal(branch1, size1);
   int totalBranch2 = calculateTotal(branch2, size2);

   printf("=== Branch Balance Totals (via Function) ===\n");
   printf("Branch 1 Total Balance : $%d\n", totalBranch1);
```

```
    printf("Branch 2 Total Balance : $%d\n", totalBranch2);


    return 0;
}
```



Q5:

```
#include <stdio.h>


// 1. Define deposit and withdraw functions
int deposit(int balance, int amount) {
    return balance + amount;
}


int withdraw(int balance, int amount) {
    return balance - amount;
}


int main() {
    int initialBalance = 30000;
    int currentBalance = initialBalance;


    int (*operationPtr)(int, int);
```

```
printf("=== Initial Account Balance : $%d ===\n\n", currentBalance);


operationPtr = deposit;
int depositAmount = 7000;


currentBalance = operationPtr(currentBalance, depositAmount);

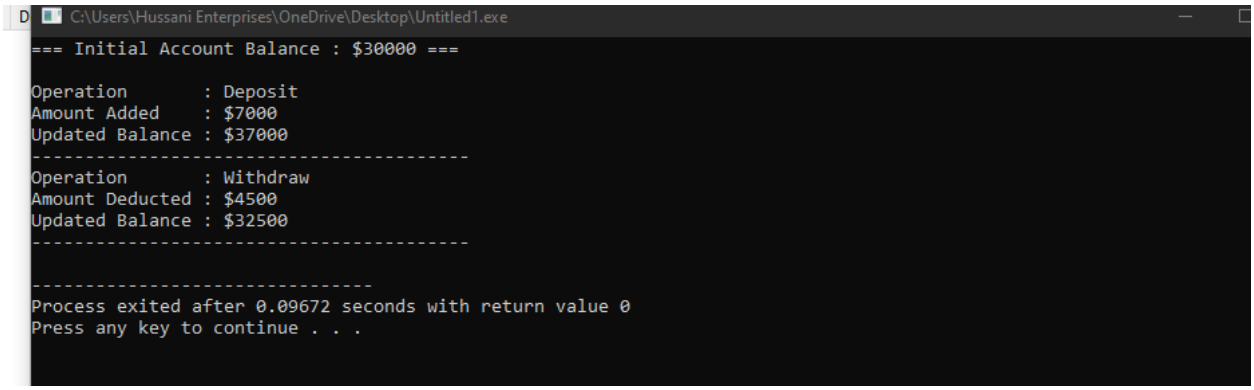

printf("Operation      : Deposit\n");
printf("Amount Added    : $%d\n", depositAmount);
printf("Updated Balance : $%d\n", currentBalance);
printf("----------------------------------------\n");


operationPtr = withdraw;
int withdrawAmount = 4500;


currentBalance = operationPtr(currentBalance, withdrawAmount);


printf("Operation      : Withdraw\n");
printf("Amount Deducted : $%d\n", withdrawAmount);
printf("Updated Balance : $%d\n", currentBalance);
printf("----------------------------------------\n");
```

```
    return 0;
}
```



Q6:

```
#include <stdio.h>

// 1. Define deposit and withdraw functions
int deposit(int balance, int amount) {
    return balance + amount;
}

int withdraw(int balance, int amount) {
    return balance - amount;
}

int processTransaction(int balance, int amount, int (*op)(int, int)) {
    return op(balance, amount);
}
```

```
int main() {
    int initialBalance = 40000;
    int currentBalance = initialBalance;

    printf("=== Initial Account Balance : $%d ===\n\n", currentBalance);

    // 3. Call processTransaction() for a deposit operation
    int depositAmount = 10000;
    currentBalance = processTransaction(currentBalance, depositAmount, deposit);

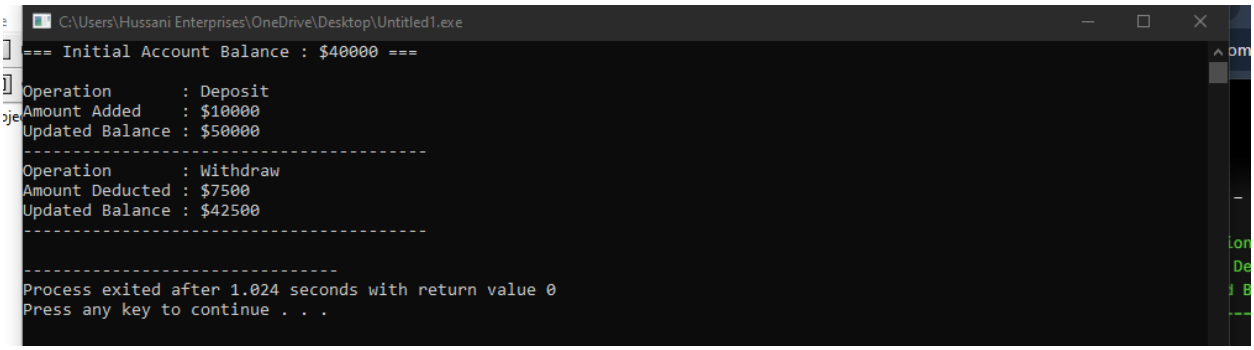
    printf("Operation      : Deposit\n");
    printf("Amount Added    : $%d\n", depositAmount);
    printf("Updated Balance : $%d\n", currentBalance);
    printf("----------------------------------------\n");

    // 4. Call processTransaction() for a withdraw operation
    int withdrawAmount = 7500;
    currentBalance = processTransaction(currentBalance, withdrawAmount, withdraw);

    printf("Operation      : Withdraw\n");
    printf("Amount Deducted : $%d\n", withdrawAmount);
    printf("Updated Balance : $%d\n", currentBalance);
    printf("----------------------------------------\n");

    return 0;
```

}


C:\Users\Hussani Enterprises\OneDrive\Desktop\Untitled1.exe
=== Initial Account Balance : $40000 ===
Operation : Deposit
Amount Added : $10000
Updated Balance : $50000
Operation : Withdraw
Amount Deducted : $7500
Updated Balance : $42500
Process exited after 1.024 seconds with return value 0
Press any key to continue . . .